

Backend Technical Documentation

Government Subsidy & Grant Disbursement Tracking System

Spring Boot Backend — Architecture, APIs, Data Model & Business Logic

Repository: [Development-of-Digital-Subsidy-and-Grant-Administration-Platform](#)

Table of Contents

1. Backend Overview
2. Backend Technology Stack
3. Backend Architecture
4. Backend Project Structure
5. Request Lifecycle
6. API Documentation
7. Controllers
8. Service Layer
9. Repository / Data Access Layer
10. Database Architecture
11. Data Models / Entities — Selected Detail
12. DTOs
13. Business Logic — Major Workflows
14. Authentication & Security
15. Configuration
16. Error Handling
17. Backend Setup
18. Database Setup
19. Testing
20. Deployment
21. Backend Maintenance Guide
22. Backend Troubleshooting

1. Backend Overview

The backend is the sole source of truth and business-rule enforcement point for the Government Subsidy & Grant Disbursement Tracking System. It is a Spring Boot 4.1.0 (Java 17) REST API that owns beneficiary registration, scheme configuration, a configurable eligibility-scoring engine, a four-stage human approval workflow, staged fund disbursement with compliance gating, officer allocation, analytics, notifications, and audit logging.

1.1 Purpose

Provide a single authoritative API that the React frontend (and, in principle, any other client) calls to register beneficiaries and officers, submit and evaluate applications, run the multi-level approval workflow, configure and release staged disbursements, and expose administrative dashboards and audit trails.

1.2 Main Responsibilities

- Authentication and role-based authorization (JWT, 6 roles).
- Beneficiary, scheme, and eligibility-rule master data management.
- Application submission and a configurable, per-scheme eligibility rule engine.
- A 4-stage verification workflow (Field → Regional → District → Finance Officer).
- Officer allocation (capacity-based, first-come-first-served, with row locking).
- Field inspection capture, tied into the workflow.
- Staged disbursement planning, sequential release, and overdue/compliance tracking with schedulers.
- File upload/storage via Cloudinary.
- Analytics/dashboard aggregation and PDF report generation.
- Real-time notifications over WebSocket plus a REST notification inbox.
- Immutable audit logging.

1.3 Framework & Language

Java 17, Spring Boot 4.1.0 (Spring Web, Spring Security, Spring Data JPA), Maven build (Maven Wrapper committed to the repo).

1.4 Architecture Style

A single deployable, layered monolith — controller → service → repository → MySQL — organised by feature package rather than split into microservices. There is no API gateway, message broker, or separate service mesh; the one exception is WebSocket-based push for notifications, layered on top of the same application.

1.5 Communication with the Frontend

The frontend communicates exclusively over HTTPS/HTTP JSON REST calls (via Axios) plus one WebSocket connection for live notifications. Authentication is carried in an HttpOnly session cookie named “token” (a JWT), not an Authorization header — Spring Security's filter chain reads the cookie on every request.

2. Backend Technology Stack

Technology	Version / Detail	Purpose
Language	Java 17	Primary implementation language
Framework	Spring Boot 4.1.0	Application framework (Spring Web, Spring Data JPA, Spring Security)
Build Tool	Maven (Maven Wrapper, mvnw)	Dependency management and build
ORM	Spring Data JPA / Hibernate	Entity mapping and repository abstraction
Database	MySQL 8 (mysql-connector-j)	Primary relational data store; ddl-auto=update
Security	Spring Security + jjwt (io.jsonwebtoken)	Authentication filter chain, password hashing, JWT issuing/validation
File Storage	Cloudinary SDK	External object storage for uploaded documents (PDF/PNG/JPG)
Real-time	Spring WebSocket (SimpMessagingTemplate / STOMP)	Push notifications to connected clients
Reporting	OpenPDF	Server-generated PDF reports (overdue milestones)
Boilerplate	Lombok 1.18.30	Getter/setter/builder code generation on entities and DTOs
Containerization	Docker (multi-stage, eclipse-temurin:17)	Build and runtime image
Testing	spring-boot-starter-test (declared, unused)	No test classes exist in the repository — see Section 19

3. Backend Architecture

The backend follows a conventional layered architecture. Every feature area (auth, scheme, application, workflow, disbursement, allocation, inspection, dashboard, report, audit, notification, media) has its own controller, service interface + implementation, and (where it owns data) repository and entities.

```

Client (React SPA)
|
|  HTTPS/JSON (Axios) + one WebSocket connection
v
JwtAuthenticationFilter (reads "token" cookie, populates SecurityContext)
|
v
Controller layer (13 @RestController classes, @PreAuthorize on 3 of them)
|
v

```

```

Service layer (interface + *Impl, business rules, @Transactional boundaries)
|
v
Repository layer (Spring Data JPA interfaces, custom @Query methods)
|
v
MySQL 8 (18 @Entity classes, ddl-auto=update)

```

Cross-cutting: `GlobalExceptionHandler` (`@RestControllerAdvice`),
`AuditLog` writes from within services, `DisbursementScheduler` (`@Scheduled`),
`CloudinaryService` (only class calling the Cloudinary SDK).

3.1 Layer Responsibilities

Layer	Responsibility
Security filter	Extracts the JWT from the “token” cookie, validates it, and populates the Spring Security context before the request reaches a controller.
Controller	HTTP concerns only — request/response mapping, path/query binding, and (in 3 of 13 controllers) declarative <code>@PreAuthorize</code> role checks.
Service	All business logic: validation, state transitions, scoring, workflow rules, disbursement math, transactional boundaries, and manual role checks where <code>@PreAuthorize</code> is absent.
Repository	Spring Data JPA interfaces; several carry custom <code>@Query</code> /derived-query methods (e.g. FCFS allocation queries with pessimistic locking).
Entity / DB	18 JPA entities mapped to MySQL tables; schema is managed by Hibernate's <code>ddl-auto=update</code> (no migration tool).

4. Backend Project Structure

```

backend/
|-- pom.xml
|-- mvnw, mvnw.cmd, .mvn/
|-- Dockerfile                (multi-stage: eclipse-temurin:17-jdk-alpine build,
|                               17-jre-alpine runtime, non-root "spring" user)
|-- docker-compose.yml        (MySQL + backend, local dev parity)
`-- src/main/
    |-- resources/
    |   |-- application.properties
    |-- java/com/example/gov_scheme_backend/
    |   |-- controllers/      13 @RestController classes
    |   |-- services/         14 service interfaces
    |   |   |-- impl/         14 *ServiceImpl classes
    |   |-- entities/         18 @Entity classes
    |   |-- repositories/     17 Spring Data JPA repositories
    |   |-- enums/            17 enums (Role, ApplicationStatus, WorkflowStage, ...)
    |   |-- dto/              request/ and response/ DTOs, one sub-package per feature
    |   |-- security/         JwtService, JwtAuthenticationFilter, CustomUserDetailsService
    |   |-- config/           SecurityConfig, CORS, Cloudinary bean
    |-- exceptions/           GlobalExceptionHandler + custom exception classes

```

There is no `src/test` directory anywhere in the backend module (confirmed by direct filesystem inspection) despite `spring-boot-starter-test` being declared in `pom.xml`.

4.1 Package Notes

- services/impl contains one file, StageApprovalServiceImpl.java, that is not referenced by any controller — a superseded 2-stage approval mechanism left in the codebase alongside the real 4-stage WorkflowService (see Section 13).
- The enums package is unusually large (17 enums) because the domain has many small closed sets: Role, ApplicationStatus, WorkflowStage, WorkflowAction, MilestoneStatus, DocumentType, NotificationType, AuditAction, Status (officer request), BeneficiaryStatus, and others.

5. Request Lifecycle

```

HTTP request arrives (cookie "token" attached automatically by the browser/Axios)
|
v
JwtAuthenticationFilter.doFilterInternal()
- No cookie / no "token" cookie -> continue unauthenticated
- Expired/invalid token         -> 401, SecurityContext cleared
- Valid token                   -> loads UserDetails, sets Authentication
|
v
Spring Security filter chain
- shouldNotFilter() bypasses the JWT filter for /gov/auth/signin,
  /gov/auth/signup, /gov/auth/officer/get-request
- SecurityConfig.authorizeHttpRequests(): anyRequest().authenticated()
|
v
Controller method
- @PreAuthorize evaluated here IF present (3 of 13 controllers)
- Otherwise the controller passes through to the service unchecked
|
v
Service method
- Manual role/ownership checks where @PreAuthorize is absent
- Business validation, state-machine checks, @Transactional writes
- AuditLog entries written on state-changing actions
|
v
Repository (Spring Data JPA) -> MySQL
|
v
Response serialized as JSON (commonly wrapped in ApiResponse{status, message}
or a feature-specific *ResponseDTO)
|
v
GlobalExceptionHandler intercepts any thrown exception along the way and
converts it to a consistent ApiResponse + HTTP status (see Section 16)

```

6. API Documentation

The API is organized under three path prefixes rather than one consistent convention: /gov/** (auth, applications, schemes, beneficiaries, workflow, notifications, audit), /api/v1/** (disbursement, allocation, dashboard, reports), and /api/** (officer inspection, media upload). This is a real inconsistency in the codebase, not a documentation simplification.

6.1 Full Endpoint Inventory (verified against controller source)

Auth — AuthController (/gov/auth)

Method	Endpoint	Controller	Auth	Purpose
GET	/gov/auth/hello	AuthController	Public	Liveness/sanity check
POST	/gov/auth/signin	AuthController	Public	Login; sets HttpOnly "token" cookie
POST	/gov/auth/signup	AuthController	Public	Beneficiary self-register OR officer request (goes to RequestsList, pending admin approval)
GET	/gov/auth/officer/get-request	AuthController	Public (filter-bypassed)	List pending officer requests
GET	/gov/auth/profile/{role}	AuthController	Authenticated, no role check in controller	List all users of a given role
PATCH	/gov/auth/approval/{uniqueId}/{status}	AuthController	Authenticated (service does not enforce ADMIN-only)	Approve/reject an officer request; creates the real Users row
GET	/gov/auth/profile/get	AuthController	Authenticated	Current user's own profile
POST	/gov/auth/signout	AuthController	Authenticated	Logout (cookie invalidation)
PUT	/gov/auth/profile/update	AuthController	Authenticated	Update own profile fields

Applications — ApplicationController (/gov/applications)

Method	Endpoint	Controller	Auth	Purpose
POST	/gov/applications/save-fields	ApplicationController	Authenticated	Save/update draft field values; runs the eligibility engine live and returns the score
POST	/gov/applications/submit/{schemeCode}	ApplicationController	Authenticated	Finalize submission for a scheme; creates the VerificationWorkflow if scoring passes
DELETE	/gov/applications/{applicationId}	ApplicationController	Authenticated (owner)	Cancel an application
PUT	/gov/applications/allocation	ApplicationController	ADMIN only (checked in controller body)	Manually allocate one application to a specific officer
POST	/gov/applications/upload-documents/{schemeCode}	ApplicationController	Authenticated	Multipart upload of supporting documents (PDF/PNG/JPG, 40MB max each)

Method	Endpoint	Controller	Auth	Purpose
GET	/gov/applications	ApplicationController	Authenticated	List applications (caller-scoped)
GET	/gov/applications/my	ApplicationController	Authenticated (officer)	List applications assigned to the current officer

Schemes — SchemeController (/gov/schemes)

Method	Endpoint	Controller	Auth	Purpose
GET	/gov/schemes/get	SchemeController	Public/Authenticated	List all schemes
GET	/gov/schemes/get-{category}	SchemeController	Public/Authenticated	List schemes filtered by category
POST	/gov/schemes/add	SchemeController	Admin (service-level check)	Create a scheme with eligibility rules, required documents, and fields
PATCH	/gov/schemes/{schemeCode}	SchemeController	Admin (service-level check)	Update a scheme

Beneficiary — BeneficiaryController (/gov/beneficiary)

Method	Endpoint	Controller	Auth	Purpose
POST	/gov/beneficiary/add	BeneficiaryController	Authenticated	Create a Beneficiary record (post-approval tracking record, separate from Users)
GET	/gov/beneficiary/get	BeneficiaryController	Authenticated	List beneficiary records
GET	/gov/beneficiary/me	BeneficiaryController	Authenticated	Current user's own beneficiary record
PUT	/gov/beneficiary/{id}	BeneficiaryController	Authenticated	Update a beneficiary record
POST	/gov/beneficiary/{id}/disburse	BeneficiaryController	Authenticated (service-level check)	Record a disbursement against the Beneficiary record
PUT	/gov/beneficiary/{id}/flag	BeneficiaryController	Authenticated (service-level check)	Flag a beneficiary record
PUT	/gov/beneficiary/{id}/unflag	BeneficiaryController	Authenticated (service-level check)	Unflag a beneficiary record

Workflow — WorkflowController (/gov/workflow)

Method	Endpoint	Controller	Auth	Purpose
POST	/gov/workflow/{applicationId}/action	WorkflowController	@PreAuthorize hasAnyRole(FIELD_OFFICER, REGIONAL_OFFICER, DISTRICT_OFFICER, FINANCE_OFFICER)	APPROVE / REJECT / ESCALATE / RE_VERIFY at the caller's current stage; Finance's APPROVE auto- creates the DisbursementPlan

Inspection — InspectionController (/api/officer)

Method	Endpoint	Controller	Auth	Purpose
GET	/api/officer/applications/{applicationId}/inspection	InspectionController	Authenticated (Field Officer)	Pre-fill context for the inspection form
POST	/api/officer/inspections/submit	InspectionController	Authenticated (Field Officer)	Submit inspection report; auto-advances the workflow to REGIONAL_OFFICER

Media — MediaController (/api/media)

Method	Endpoint	Controller	Auth	Purpose
POST	/api/media/upload	MediaController	Authenticated	Upload a single evidence image (used for inspection evidence)

Allocation — AllocationController (/api/v1/allocation)

Method	Endpoint	Controller	Auth	Purpose
GET	/api/v1/allocation/officers/available	AllocationController	Admin	Officer workload/capacity for a given workflow stage
POST	/api/v1/allocation/batch	AllocationController	Admin	FCFS batch-allocate N pending applications at a stage, row-locked
GET	/api/v1/allocation/summary	AllocationController	Admin	Allocation queue summary across stages
GET	/api/v1/allocation/officers/capacity	AllocationController	Admin	Per-officer capacity for a stage

Method	Endpoint	Controller	Auth	Purpose
PUT	/api/v1/allocation/officers/{officerId}/capacity	AllocationController	Admin	Update an officer's allocation capacity

Disbursement — DisbursementController (/api/v1/disbursement)

Method	Endpoint	Controller	Auth	Purpose
POST	/api/v1/disbursement/plan/{planId}/configure	DisbursementController	Finance Officer (service-level check)	Configure milestone stages; amounts must sum exactly to the plan total
POST	/api/v1/disbursement/milestone/{milestoneId}/submit-proof	DisbursementController	Authenticated (beneficiary)	Submit compliance proof documents for a milestone
POST	/api/v1/disbursement/milestone/{milestoneId}/reject-proof	DisbursementController	Officer (service-level check)	Reject submitted proof, requests resubmission
POST	/api/v1/disbursement/milestone/{milestoneId}/complete	DisbursementController	Officer/Admin (service-level check)	Mark a milestone COMPLETED after reviewing proof
POST	/api/v1/disbursement/release/{milestoneId}	DisbursementController	Finance Officer (service-level check)	Release funds for a COMPLETED milestone; sequential-gate + check-then-act double-release guard
GET	/api/v1/disbursement/plan/application/{applicationId}	DisbursementController	Authenticated	Fetch the disbursement plan for an application
GET	/api/v1/disbursement/schemes/{schemeCode}	DisbursementController	Authenticated	Scheme-scoped disbursement info
GET	/api/v1/disbursement/audit-logs	DisbursementController	Admin	Disbursement-related audit trail

Method	Endpoint	Controller	Auth	Purpose
PUT	/api/v1/disbursement/milestone/{milestoneId}/resolve	DisbursementController	Admin (service-level check)	Admin override to clear an OVERDUE milestone (reason required)
GET	/api/v1/disbursement/notifications	DisbursementController	Authenticated	Disbursement-specific reminder/overdue notification feed
POST	/api/v1/disbursement/seed	DisbursementController	Authenticated only — no role restriction found	Seeds demo/test data — appears to be a development leftover
POST	/api/v1/disbursement/overdue/check	DisbursementController	Authenticated	Manually trigger the overdue-flagging scheduler logic
POST	/api/v1/disbursement/reminders/check	DisbursementController	Authenticated	Manually trigger the upcoming-reminder scheduler logic
GET	/api/v1/disbursement/plan/{planId}/suggest-stages	DisbursementController	Authenticated	Suggests an even stage split for a plan
GET	/api/v1/disbursement/milestone/{milestoneId}/context	DisbursementController	Authenticated	Milestone detail for UI display

Dashboard — DashboardController (/api/v1/dashboard)

Method	Endpoint	Controller	Auth	Purpose
GET	/api/v1/dashboard/schemes	DashboardController	Authenticated	Per-scheme allocated/used/remaining funds and utilization %, real SQL aggregation
GET	/api/v1/dashboard/regions	DashboardController	Authenticated	Application counts by region (no fund totals by region)

Method	Endpoint	Controller	Auth	Purpose
GET	/api/v1/dashboard/performance	DashboardController	Authenticated	Totals by status, avg approval/disbursement days, missing-docs %
GET	/api/v1/dashboard/categories	DashboardController	Authenticated	Fund breakdown by scheme category
GET	/api/v1/dashboard/trends	DashboardController	Authenticated	Monthly trend series
GET	/api/v1/dashboard/sparkline	DashboardController	Authenticated	Sparkline series for KPI cards
GET	/api/v1/dashboard/officer-queue	DashboardController	Authenticated	Officer queue depth
GET	/api/v1/dashboard/flag-reasons	DashboardController	Authenticated	Breakdown of flagged-beneficiary reasons
GET	/api/v1/dashboard/rejection-reasons	DashboardController	Authenticated	Breakdown of application rejection reasons

Reports — ReportController (/api/v1/reports)

Method	Endpoint	Controller	Auth	Purpose
GET	/api/v1/reports/overdue	ReportController	Authenticated	List overdue milestones
POST	/api/v1/reports/overdue/check	ReportController	Authenticated	Manual trigger, report-side
GET	/api/v1/reports/overdue/pdf	ReportController	Authenticated	Download the overdue-milestones report as a PDF (OpenPDF)

Notifications — NotificationController (/gov/notifications)

Method	Endpoint	Controller	Auth	Purpose
GET	/gov/notifications	NotificationController	@PreAuthorize authenticated	Current user's notifications
GET	/gov/notifications/unread-count	NotificationController	@PreAuthorize authenticated	Unread badge count
PATCH	/gov/notifications/{notificationId}/read	NotificationController	@PreAuthorize authenticated	Mark one notification read
PATCH	/gov/notifications/read-all	NotificationController	@PreAuthorize authenticated	Mark all read

Audit — AuditController (/gov/audit)

Method	Endpoint	Controller	Auth	Purpose
GET	/gov/audit	AuditController	@PreAuthorize hasRole(ADMIN)	List all audit log entries — read-only, no update/delete endpoint exists (immutability by omission)

6.2 Representative Request/Response Examples

POST /gov/auth/signin

Request

```
{
  "username": "beneficiary_user1",
  "password": "P@ssw0rd1"
}
```

Response (200)

```
// HTTP 200, Set-Cookie: token=<jwt>; HttpOnly
{
  "status": true,
  "message": "<JWT string>"
}
```

Processing flow: AuthController delegates to AuthServiceImpl.loginService(), which authenticates via Spring's AuthenticationManager, writes a LOGIN AuditLog entry for any non-beneficiary role, then issues a JWT (JwtService.generateToken) containing role and userId claims with a 30-minute expiry, returned as an HttpOnly cookie.

POST /gov/applications/save-fields

Request

```
{
  "schemeCode": "PM-KISAN-01",
  "fields": {
    "ANNUAL_INCOME": 85000,
    "OCCUPATION": "Farmer",
    "LAND_AREA": 2.5
  }
}
```

Response (200)

```
{
  "applicationId": 42,
  "score": 78.0,
  "totalPossibleScore": 100.0,
  "passed": true
}
```

Processing flow: ApplicationServiceImpl.saveFields() upserts the draft ApplicationFieldValue rows, then calls EligibilityEngineServiceImpl.validateFields(), which evaluates every SchemeEligibilityRule configured for that scheme (operators: EQUALS, NOT_EQUALS, GREATER_THAN, GREATER_THAN_EQUAL, LESS_THAN, LESS_THAN_EQUAL, with partial-credit tolerance bands) and returns a live score. The score is not persisted as a separate historical record — it is recomputed on every save (see Section 13).

POST /gov/workflow/{applicationId}/action

Request

```
{
  "action": "APPROVE",
  "approvedAmount": 25000,
  "numberOfInstallments": 3,
  "remarks": "All documents verified"
}
```

Processing flow: `WorkflowServiceImpl.processAction()` calls `validateOfficer()` (checks the assigned officer matches the caller AND the caller's role matches the current `WorkflowStage`), applies the action, writes a `WorkflowHistory` row and an `AuditLog` entry, and — only when the action is `APPROVE` at the `FINANCE_OFFICER` stage — calls `createDisbursementPlan()`, which sets `DisbursementPlan.totalAmount` from `approvedAmount` with no validation against any `grant_min/grant_max` bound (no such fields exist on `Schemes`) and no check against remaining scheme budget at this point (that check only happens later, at milestone release).

POST `/api/v1/disbursement/release/{milestoneId}`

Processing flow: `DisbursementServiceImpl.releaseMilestone()` checks the milestone is `COMPLETED` and not already `RELEASED` (a check-then-act comparison — `DisbursementMilestone` has no `@Version` column, so this is not protected by optimistic locking or a conditional `UPDATE`), checks sequential ordering (stage N requires stage N-1 released and not `OVERDUE`), then atomically updates milestone status, `amountReleased`, release date, `scheme.budgetUsed`, and writes an `AuditLog` entry inside a single `@Transactional` method.

7. Controllers

Controller	Base Path	Responsibility
<code>AuthController</code>	<code>/gov/auth</code>	Signup/login/logout, officer approval queue, profile get/update. No role restriction on approving officer requests at the controller/security level — enforced only informally.
<code>ApplicationController</code>	<code>/gov/applications</code>	Draft save, submission, document upload, cancellation, manual allocation. Reads the JWT cookie directly in several methods (via <code>JwtService</code>) rather than relying purely on the security context.
<code>SchemeController</code>	<code>/gov/schemes</code>	Scheme CRUD and category-filtered listing, including nested eligibility rules, required documents, and dynamic fields.
<code>BeneficiaryController</code>	<code>/gov/beneficiary</code>	CRUD plus flag/unflag and disburse actions on the separate <code>Beneficiary</code> tracking entity.
<code>WorkflowController</code>	<code>/gov/workflow</code>	The single action endpoint that drives the entire 4-stage approval state machine. The only controller besides <code>Audit/Notification</code> using <code>@PreAuthorize</code> .
<code>InspectionController</code>	<code>/api/officer</code>	Field inspection context and submission; submission auto-advances the workflow.
<code>MediaController</code>	<code>/api/media</code>	Generic single-file upload wrapper around <code>CloudinaryService</code> , used by the inspection-evidence flow.
<code>AllocationController</code>	<code>/api/v1/allocation</code>	Officer workload queries and FCFS batch/individual allocation with capacity management.
<code>DisbursementController</code>	<code>/api/v1/disbursement</code>	The largest controller (958-line service behind it): plan configuration, proof submission/rejection/completion, release, resolve, scheduler manual triggers, and a seed endpoint.
<code>DashboardController</code>	<code>/api/v1/dashboard</code>	Nine read-only analytics endpoints, each backed by real aggregate queries.

Controller	Base Path	Responsibility
ReportController	/api/v1/reports	Overdue-milestone listing and PDF export.
NotificationController	/gov/notifications	REST inbox on top of the WebSocket push mechanism.
AuditController	/gov/audit	Read-only audit trail, ADMIN-only, no mutation endpoints.

8. Service Layer

8.1 EligibilityEngineServiceImpl

Implements a generic, per-scheme configurable rule engine rather than the fixed “Income 30 + Category 40 + Documents 30” formula described in the original requirements plan. Rules are evaluated against a fixed set of applicant fields (AGE, ANNUAL_INCOME, LAND_AREA, OCCUPATION, CASTE, STATE, GENDER) using EQUALS / NOT_EQUALS / GREATER_THAN / GREATER_THAN_EQUAL / LESS_THAN / LESS_THAN_EQUAL, with tolerance bands that award partial credit for near-misses on numeric rules. The score is a live computation, never persisted as a separate historical record (no eligibility_score_log-equivalent entity exists).

8.2 WorkflowServiceImpl

The real approval engine. Stages, in order: FIELD_OFFICER → REGIONAL_OFFICER → DISTRICT_OFFICER → FINANCE_OFFICER → COMPLETED. Actions: APPROVE, REJECT, ESCALATE, RE_VERIFY. validateOfficer() checks both that the caller is the officer assigned to the application and that the caller's role matches the current stage. Every action writes WorkflowHistory and AuditLog rows. The Finance stage's APPROVE additionally validates and persists the approved amount/installment count and creates the DisbursementPlan.

8.3 DisbursementServiceImpl (largest service, ~960 lines)

- configurePlan(): stage amounts must sum exactly to the plan total; stage 1 is auto-released immediately on configuration.
- Sequential gating: a stage cannot be released until the previous stage is RELEASED and not OVERDUE.
- Milestone status flow observed in code: PENDING → PROOF_SUBMITTED → COMPLETED → RELEASED, with a PROOF_REJECTED branch back to PENDING — richer than the PENDING/COMPLETED/RELEASED/OVERDUE flow described in the requirements plan.
- resolveMilestone(): admin override for an OVERDUE milestone, reason required, logged.
- Budget check (scheme.budgetUsed + amount <= scheme.allocatedFunds) is enforced at release time, not at plan-creation time.
- Double-release protection is a check-then-act status comparison; DisbursementMilestone has no @Version column, so there is no database-level guard against a race condition on concurrent release calls.

8.4 AllocationServiceImpl

`getAvailableOfficers()` computes remaining capacity per officer for a given `WorkflowStage` (default capacity 10). `batchAllocate()` is a first-come-first-served assignment: it locks the oldest unassigned workflow rows for a stage (`findOldestUnassignedWorkflowsByStageWithLock`, a pessimistic-locking query), then assigns each to the officer with the lowest current load who still has capacity, publishing a notification and an `AuditLog` entry per assignment. This allocation logic does not filter by district or region at all, even though `Users` carries `district/region/state` fields — the requirements plan's "Field Officer queues filtered by `assigned_district`" is not implemented.

8.5 FieldInspectionServiceImpl

`submitInspectionReport()` persists a `FieldInspection` row and, in the same `@Transactional` method, calls `WorkflowService.processAction()` with an `APPROVE` action to move the application from `FIELD_OFFICER` to `REGIONAL_OFFICER` — the Field Officer's role is "inspect and forward," not approve/reject.

8.6 AuthServiceImpl

Handles both beneficiary self-registration (direct `Users` row) and officer registration (goes into a `RequestsList` row with `PENDING` status, requiring a separate admin approval call before a real `Users` login exists). Password policy is enforced by regex (starts uppercase, requires lowercase/digit/special character, minimum length 7). No duplicate-username pre-check exists in `signupService` — the database's unique constraint on username is the only backstop, so a duplicate signup surfaces as a `DataIntegrityViolationException` rather than a friendly validation message. `deleteProfile()` exists in this service with no corresponding controller endpoint (see Section 6).

8.7 CloudinaryServiceImpl

The only class in the codebase that calls the Cloudinary SDK. Validates content type (PDF/PNG/JPG only) and size (40MB max) before upload; returns the `secure_url` for storage in `ApplicationDocument.documentUrl` (or the milestone-linked equivalent). `deleteFile()` exists and is called by other services when a document is replaced; failures to delete are logged, not thrown, since the DB record cleanup is treated as the primary concern.

8.8 DashboardServiceImpl

Backs all nine dashboard endpoints with real aggregate queries (grouped counts, sums, and computed percentages) rather than static or mocked figures. `getRegionDashboard()` only returns application counts by region — it does not include fund totals by region, which is a narrower scope than the requirements plan's "region-wise dashboard: applications/funds by district." No explicit 80%-budget-exhaustion flag or compliance-rate (% milestones on time vs. overdue) metric is computed anywhere in this service.

8.9 NotificationServiceImpl

Combines a REST-queryable notification inbox with WebSocket push (`SimpMessagingTemplate`). Notification creation is deliberately registered to fire only after the enclosing transaction commits (`TransactionSynchronizationManager.registerSynchronization(...).afterCommit()`), so a rolled-back business operation never produces a stray notification.

8.10 StageApprovalServiceImpl — not part of the active system

Implements an independent 2-stage (FIELD → DISTRICT) approval flow using its own ReviewStage enum. Confirmed by repository-wide search: no controller references this service. It appears to be an earlier design that was superseded by the 4-stage WorkflowService but was never deleted.

9. Repository / Data Access Layer

17 Spring Data JPA repository interfaces, one per major entity, using a mix of derived query methods (findByUsername, findByRole, countByStatus, etc.) and explicit @Query methods for aggregate/analytics queries and locking reads.

9.1 Notable custom queries

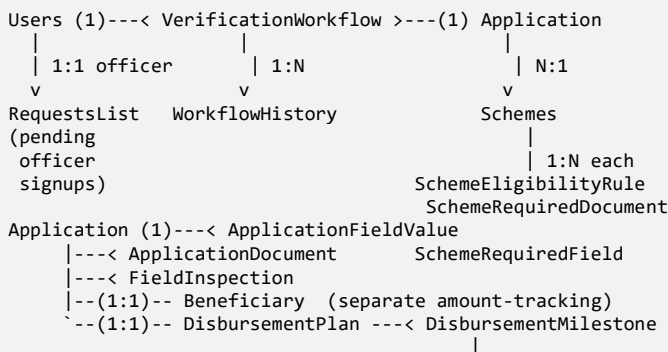
- VerificationWorkflowRepository.findOldestUnassignedWorkflowsByStageWithLock(stage, pageable) — a pessimistic-locking, oldest-first query that underpins FCFS batch allocation.
- VerificationWorkflowRepository.countActiveAssignmentsByOfficer(officerId) — used to compute live officer workload for capacity checks.
- ApplicationRepository.countApplicationsByScheme() / countApplicationsByRegion() / getApplicationPerformance() / countByDocumentsIsEmpty() — the aggregate queries behind the analytics dashboards.
- MilestoneRepository.countByCompletionStatus(status) — used for the “flagged milestones” KPI and the overdue scheduler.

No repository uses native SQL; all custom queries are JPQL. No pagination is applied to most list endpoints beyond the allocation batch query (PageRequest is used there specifically to cap how many workflow rows are locked per call).

10. Database Architecture

MySQL 8, accessed via Spring Data JPA/Hibernate with ddl-auto=update (schema is generated/alterred automatically from the entity classes — there is no Flyway/Liquibase migration tool and no versioned schema history).

10.1 Entity Relationship Overview



<pre> `---< document (proof; ApplicationDocument.milestoneId) Users (1)---< Notification Users/Application actions ---< AuditLog (immutable, no update/delete) </pre>

10.2 Entities (18 total) — key fields and notes

Entity	Key Fields	Notes
Users	id, uniqueID, fullName, username (unique), password, role, region, district, state, allocationCapacity	district/region exist but are not used to filter officer allocation anywhere
RequestsList	uniqueID, username, role, status (PENDING/APPROVED/REJECTED)	Holding table for officer signups pending admin approval
Schemes	schemeCode, schemeName, benefit, allocatedFunds, budgetUsed, minimumEligibleScore, active	No grant_min/grant_max fields exist
SchemeEligibilityRule	schemeId (FK), fieldName, operator, expectedValue, partialPercentage	Drives the eligibility engine
SchemeRequiredDocument	schemeId (FK), documentType	Scheme-configurable required document list
SchemeRequiredField	schemeId (FK), fieldName, mandatory	Drives the dynamic application form on the frontend
Application	id, applicationCode, user (FK), scheme (FK), status, currentStage, allocatedOfficer (FK)	status is ApplicationStatus enum; includes an unused INSPECTION_COMPLETED value
ApplicationFieldValue	applicationId (FK), fieldName, value	Draft/submitted field answers, scored live
ApplicationDocument	applicationId (FK, nullable), milestoneId (FK, nullable), documentType, documentUrl	documentUrl stores the Cloudinary secure_url; shared between initial application docs and milestone compliance proof
VerificationWorkflow	applicationId (FK), stage, assignedOfficer (FK)	Core workflow-state table
WorkflowHistory	workflowId (FK), action, actor (FK), remarks, timestamp	Append-only action trail per workflow
FieldInspection	applicationId (FK), officer (FK), addressVerified, businessActivityConfirmed, assetsInspected, notes, evidenceMediaIds, submittedAt	Backs the Field Officer's inspection report
Beneficiary	user (FK), application (FK, unique), sanctionedAmount (Double), disbursedAmount (Double), currentStatus, isFlagged, flagReason	Money fields are Double here, inconsistent with BigDecimal used elsewhere

Entity	Key Fields	Notes
DisbursementPlan	applicationId (FK), totalAmount (BigDecimal), totalStages, numberOfInstallments	Created automatically on Finance's final APPROVE
DisbursementMilestone	planId (FK), stageNumber, amount (BigDecimal), dueDate, completionStatus, amountReleased, releaseDate	No @Version column — no optimistic locking
Notification	userId (FK), message, type, readStatus, createdAt	Backs both the REST inbox and WebSocket push payloads
AuditLog	auditId, user (FK), action (enum), description, createdAt	Free-text description field; no separate old_value/new_value columns

11. Data Models / Entities — Selected Detail

11.1 Users

Implements Spring Security's UserDetails directly (isAccountNonExpired/isAccountNonLocked/isCredentialsNonExpired/isEnabled all hardcoded to true — there is no account-locking or expiry mechanism). getAuthorities() returns a single ROLE_<Role> authority per user; the system does not support multiple roles per account.

11.2 Schemes

allocatedFunds and budgetUsed are the only budget-tracking fields; there is no minimum/maximum grant amount per scheme. minimumEligibleScore is compared against the eligibility engine's score, capped defensively at the rule set's totalPossibleScore so a scheme with a low total possible score can't produce an unreachable threshold.

11.3 DisbursementMilestone

completionStatus follows PENDING → PROOF_SUBMITTED → COMPLETED → RELEASED, with PROOF_REJECTED and OVERDUE as side-branches. amountReleased and releaseDate are set only inside the single @Transactional releaseMilestone() method alongside the scheme's budgetUsed update.

11.4 AuditLog

Structurally simple: an enum action, a free-text description, the acting user, and a timestamp. There are no old_value/new_value columns, so "what changed" is only as detailed as the human-readable description string each service chooses to write. No controller exposes update or delete for this entity.

12. DTOs

Request and response DTOs are organized into dto/request/<feature> and dto/response/<feature> sub-packages, generally one class per endpoint or per meaningful response shape (e.g. dto/request/auth/{LoginRequest,SignupRequest}, dto/response/application/EligibilityEngineScoreDTO,

dto/request/workflow/WorkflowActionRequest,
 dto/response/allocation/{OfficerWorkloadDTO,BatchAllocationResponseDTO,ApplicationAllocationResponseDTO}, dto/response/dashboard/* for each of the nine analytics shapes).

A common ApiResponse{boolean status, String message} wrapper is used for simple success/failure endpoints (auth, profile management); feature-rich endpoints (workflow, disbursement, dashboard, allocation) return dedicated response DTOs instead. Bean Validation annotations were not found in wide use — most input validation is manual (regex checks in AuthServiceImpl, null/blank checks) rather than declared via @NotNull/@Valid, which is why MethodArgumentNotValidException appears in GlobalExceptionHandler but is not the primary validation path in practice.

13. Business Logic — Major Workflows

13.1 Registration → Application → Scoring

1. Beneficiary signs up (POST /gov/auth/signup) -> Users row created directly.
2. Beneficiary saves draft fields for a scheme (POST /gov/applications/save-fields)
 -> EligibilityEngineServiceImpl scores live against SchemeEligibilityRule rows.
3. Beneficiary uploads required documents (POST /gov/applications/upload-documents/{schemeCode}).
4. Beneficiary submits (POST /gov/applications/submit/{schemeCode})
 -> if status is not DRAFT/PENDING, a DuplicateResourceException is thrown —
 this also blocks re-application after a REJECTED outcome, which differs
 from the original requirements plan's intent to allow reapplication
 after a terminal rejection.
 -> on success, a VerificationWorkflow row is created at stage FIELD_OFFICER.

13.2 Approval Workflow

```
FIELD_OFFICER -- inspects, submits FieldInspection, auto-forwards (no explicit
                  approve/reject choice at this stage) --> REGIONAL_OFFICER
REGIONAL_OFFICER -- APPROVE/REJECT/ESCALATE/RE_VERIFY --> DISTRICT_OFFICER
DISTRICT_OFFICER -- APPROVE/REJECT/ESCALATE/RE_VERIFY --> FINANCE_OFFICER
FINANCE_OFFICER -- APPROVE (sets approvedAmount + numberOfInstallments,
                        auto-creates DisbursementPlan) / REJECT --> COMPLETED
```

Every transition is guarded by validateOfficer() (assigned officer + role/stage match) and writes both a WorkflowHistory row and an AuditLog entry. RE_VERIFY returns the application to a re-submittable state; the beneficiary's resubmission re-runs the eligibility engine.

13.3 Disbursement

```
Finance APPROVE --> DisbursementPlan created (totalAmount, no grant-range check)
Finance configures stages (sum must equal totalAmount) --> stage 1 auto-released
Beneficiary submits proof for a stage --> PROOF_SUBMITTED
Officer/Admin reviews --> COMPLETED or PROOF_REJECTED (back to PENDING)
Finance releases a COMPLETED, non-OVERDUE stage whose predecessor is RELEASED
--> budget check against scheme.allocatedFunds/budgetUsed happens HERE,
    not at plan-creation time
Daily schedulers (no explicit timezone set) flag OVERDUE and send reminders;
an OVERDUE stage blocks the next release until an Admin resolves it.
```

13.4 Officer Allocation

Two allocation paths exist: (1) AllocationController's FCFS batch endpoint, which locks the oldest unassigned workflow rows for a stage and assigns them to the least-loaded officer with remaining

capacity, and (2) ApplicationController's PUT /allocation, which lets an Admin manually assign one specific application to one specific officer. Neither path filters by district/region.

14. Authentication & Security

14.1 Authentication Mechanism

JWT-based, but the token is delivered and read as an HttpOnly cookie named "token" rather than an Authorization header. JwtService.generateToken() embeds role and userId claims and sets a fixed 30-minute expiry. JwtAuthenticationFilter reads the cookie on every request, validates signature and expiry, and populates the Spring Security context. Requests to /gov/auth/signin, /gov/auth/signup, and /gov/auth/officer/get-request bypass the filter (shouldNotFilter).

14.2 Password Handling

Passwords are hashed with Spring Security's PasswordEncoder before storage. Signup enforces a password policy via regex: must start with an uppercase letter and contain a lowercase letter, a digit, a special character, and be at least 7 characters total.

14.3 Authorization (RBAC)

Six roles: BENEFICIARY, FIELD_OFFICER, REGIONAL_OFFICER, DISTRICT_OFFICER, FINANCE_OFFICER, ADMIN. Officer accounts require an admin-approval step (RequestsList) before a functioning login exists.

Role enforcement is not uniform across the API. @PreAuthorize is used in exactly 3 of 13 controllers (Audit, Notification, Workflow). Everywhere else — Disbursement, Application, Allocation, Scheme, Beneficiary, Inspection, Report, Media — role checks are written manually inside service methods (or, in a couple of ApplicationController methods, inline in the controller by manually decoding the JWT claims), rather than declared once at the entry point. This makes it easy to add a new endpoint to one of these controllers without any authorization check at all, since there's no framework-level default-deny in place beyond "must be authenticated."

14.4 CORS

SecurityConfig configures CorsConfigurationSource with allowedOriginPatterns("*") and allowCredentials(true). This is a broad configuration — it accepts requests from any origin pattern while also allowing credentialed (cookie-bearing) requests, rather than being scoped to the frontend's specific deployed domain.

14.5 CSRF

Not explicitly configured in the reviewed SecurityConfig beyond Spring Security's defaults for a stateless JWT-cookie setup; no CSRF token mechanism was found. Not identified in the repository as either enabled or explicitly disabled with a documented rationale.

14.6 Session/Token Expiry

30 minutes, hardcoded in JwtService (no refresh-token mechanism). An expired token causes the filter to clear the security context and return 401; the frontend's Axios interceptor catches this and redirects to /login (see Frontend Documentation, Section 10).

14.7 Security Filter Chain Summary

Path pattern	Access rule
/gov/auth/signin, /gov/auth/signup, /gov/auth/officer/get-request	Bypasses the JWT filter entirely (public)
/gov/audit/**	@PreAuthorize hasRole('ADMIN')
/gov/notifications/**	@PreAuthorize authenticated
/gov/workflow/**	@PreAuthorize hasAnyRole(FIELD_OFFICER, REGIONAL_OFFICER, DISTRICT_OFFICER, FINANCE_OFFICER)
Everything else	anyRequest().authenticated() at the filter-chain level; role granularity (if any) is enforced manually inside the service or controller method

14.8 Confirmed Security Concerns

- application.properties ships real-looking default values (fallbacks used when the corresponding environment variable is not set) for the MySQL password, the JWT signing secret, and the Cloudinary cloud name/API key/API secret. These are present as hardcoded fallbacks in a public repository, independent of whether they are still live in any deployed environment. Immediate rotation of all of these credentials is recommended regardless.
- CORS accepts any origin pattern with credentials enabled, rather than being scoped to the deployed frontend domain.
- POST /api/v1/disbursement/seed requires only authentication, not a specific role — any logged-in user (including a BENEFICIARY) could invoke it if the frontend does not gate the button, since there is no server-side role check visible on this endpoint.
- Inconsistent @PreAuthorize coverage (Section 14.3) means authorization correctness for most of the API depends on every relevant service method remembering to check the caller's role manually.

15. Configuration

All configuration lives in a single application.properties file. Every value is environment-variable-overridable, but each also ships a default fallback (visible directly in the committed file) that will be used if the corresponding variable is not set at runtime.

Property	Environment Variable	Purpose
spring.datasource.url	DB_URL	MySQL JDBC connection string
spring.datasource.username	DB_USERNAME	Database user

Property	Environment Variable	Purpose
spring.datasource.password	DB_PASSWORD	Database password — a real-looking default is committed; rotate and never rely on the fallback
spring.jpa.hibernate.ddl-auto	— (hardcoded to update)	Schema auto-managed by Hibernate; no migration tool
server.port	PORT	HTTP port, defaults to 8080
spring.servlet.multipart.max-file-size / max-request-size	—	40MB / 100MB upload limits
jwt.secret	JWT_SECRET	HMAC signing key for JWTs — a real-looking default is committed; rotate
cloudinary.cloud-name / api-key / api-secret	CLOUDINARY_CLOUD_NAME / CLOUDINARY_API_KEY / CLOUDINARY_API_SECRET	Cloudinary credentials for document storage — real-looking defaults are committed; rotate

15.1 Logging

spring.jpa.show-sql=true and hibernate.format_sql=true are enabled, meaning generated SQL is logged — appropriate for development, worth reviewing before a production cutover since it increases log volume and can incidentally surface query parameter values in logs.

16. Error Handling

A single `@RestControllerAdvice` (`GlobalExceptionHandler`) converts exceptions to a consistent `{ status: false, message: "..."` } `ApiResponse` shape with an appropriate HTTP status.

Exception	HTTP Status	Notes
ResourceNotFoundException	404	Custom exception for missing entities
DuplicateResourceException	409	Custom exception, e.g. duplicate Aadhaar or a re-submission blocked by application status
BadRequestException	400	Custom exception for validation failures raised explicitly in service code
MethodArgumentNotValidException	400	Bean Validation failures, field errors joined into one message
MissingServletRequestParameterException	400	Missing required query parameter
HttpMessageNotReadableException	400	Malformed JSON body
IllegalArgumentException	400	Generic bad-argument fallback
BadCredentialsException	401	Login failure
UsernameNotFoundException	401	Login failure (unknown username)

Exception	HTTP Status	Notes
DataIntegrityViolationException	Handled (status code not fully confirmed in the excerpt reviewed)	Covers DB-level constraint violations, e.g. duplicate username on signup, since no pre-check exists in AuthServiceImpl

17. Backend Setup

17.1 Prerequisites

- Java 17 JDK
- MySQL 8 (local instance, or via the provided docker-compose.yml)
- Maven Wrapper (committed — no separate Maven install required)

17.2 Environment Variables

Set before running in any environment other than local dev with the defaults: DB_URL, DB_USERNAME, DB_PASSWORD, JWT_SECRET, CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET, PORT (optional, defaults to 8080).

17.3 Running Locally

```
cd backend
./mvnw spring-boot:run
# or, to build a jar first:
./mvnw clean package -DskipTests
java -jar target/gov-scheme-backend-0.0.1-SNAPSHOT.jar
```

17.4 Running via Docker

```
cd backend
docker build -t gov-scheme-backend .
docker run -p 8080:8080 \
  -e DB_URL=... -e DB_USERNAME=... -e DB_PASSWORD=... \
  -e JWT_SECRET=... \
  -e CLOUDINARY_CLOUD_NAME=... -e CLOUDINARY_API_KEY=... -e CLOUDINARY_API_SECRET=... \
  gov-scheme-backend

# or, for local dev parity with MySQL included:
docker compose up
```

The Dockerfile is a two-stage build (eclipse-temurin:17-jdk-alpine to compile, eclipse-temurin:17-jre-alpine to run) and drops privileges to a non-root "spring" user in the runtime stage.

18. Database Setup

There is no separate schema-initialization script and no migration tool (Flyway/Liquibase not present). spring.jpa.hibernate.ddl-auto=update means Hibernate creates/alters tables automatically from the entity classes on application startup against whatever database DB_URL points to. There is no

committed seed-data SQL script; the closest equivalent is the POST `/api/v1/disbursement/seed` endpoint, which appears to programmatically create demo data at runtime rather than via a SQL file.

Note: Because `ddl-auto=update` has no rollback story and no migration history, schema changes in a shared/production database should be reviewed carefully before deploying a new backend version.

19. Testing

Confirmed directly: no `src/test` directory exists anywhere under `backend/`, and no test classes of any kind were found in the repository, despite `spring-boot-starter-test` being declared as a dependency in `pom.xml`. There is no CI configuration file (no `.github/workflows`, no equivalent) found in the repository. Not identified in the repository: any test framework actually in use, any test coverage, any test-running command that would succeed.

20. Deployment

20.1 Configured for deployment (confirmed in the repository)

- Dockerfile — multi-stage build producing a runnable jar image, non-root user, exposes port 8080.
- `docker-compose.yml` — backend + MySQL for local development parity.
- Environment-variable-driven configuration for DB, JWT secret, and Cloudinary credentials, suitable for injection by a PaaS.

20.2 Actually deployed (unverified from the repository alone)

The frontend's README/comments reference a Render-hosted backend URL (<https://govt-subsidy-disbursement-tracking-system.onrender.com>), but this could not be independently confirmed as live from the repository contents, and the frontend's own Axios client hardcodes `http://localhost:8080` as its `baseUrl` rather than reading this URL from configuration (see Frontend Documentation, Section 10). Whether the production deployment is currently functional is Not identified in the repository — it would need to be checked directly against the live URL, which is outside the scope of a source-code review.

21. Backend Maintenance Guide

Task	Where to make the change
Add a new API endpoint	Add a method to the relevant controller in <code>controllers/</code> , delegate to a new or existing method in the matching <code>services/</code> interface + impl
Add a new database entity	Add an <code>@Entity</code> class in <code>entities/</code> , a matching <code>JpaRepository</code> in <code>repositories/</code> , then reference it from the owning service; <code>ddl-auto=update</code> will create the table on next startup
Add a new business rule	Almost always belongs in the relevant <code>*ServiceImpl</code> (e.g. new eligibility operator in <code>EligibilityEngineServiceImpl</code> , new workflow action in <code>WorkflowServiceImpl</code>)

Task	Where to make the change
Modify an existing endpoint's behavior	Change the service method it delegates to, not the controller, unless the change is to request/response shape (DTOs) or HTTP semantics
Add request validation	Prefer adding explicit checks in the service (matching the codebase's existing style) or Bean Validation annotations on the DTO plus <code>@Valid</code> on the controller parameter
Add a new role-restricted endpoint	Add <code>@PreAuthorize</code> on the controller method for consistency with Audit/Notification/Workflow, rather than only checking the role inside the service, given the coverage gap noted in Section 14.3
Add a new database query	Add a derived method name or an <code>@Query</code> to the relevant repository interface in repositories/
Change JWT expiry or claims	security/JwtService.java
Change CORS policy	config/SecurityConfig.java

22. Backend Troubleshooting

Symptom	Likely Cause / Fix
App fails to start with a datasource connection error	MySQL is not reachable at the configured DB_URL, or DB_USERNAME/DB_PASSWORD are wrong — check docker-compose.yml or your local MySQL instance
401 on every request after ~30 minutes of use	JWT expiry (hardcoded 30 minutes in JwtService) — the user needs to log in again; there is no refresh-token flow
A newly added controller endpoint is reachable by any authenticated user regardless of role	Expected given the current inconsistent authorization pattern (Section 14.3) — add a manual role check in the service or a <code>@PreAuthorize</code> annotation on the controller method
File upload fails with a 400	Check the file's content type (only PDF/PNG/JPG are accepted) and size (40MB cap) in CloudinaryServiceImpl.validateFile()
A signup with an already-used username returns a generic 500-style error instead of a friendly message	There is no pre-check for duplicate usernames in AuthServiceImpl.signupService(); the database unique constraint is the only backstop and surfaces as DataIntegrityViolationException
Milestone release appears to double-release under concurrent clicks in testing	Known gap: no <code>@Version</code> /optimistic locking on DisbursementMilestone; the guard is a check-then-act comparison only (Section 8.3)
Schema changes don't appear as expected in a shared database	ddl-auto=update does not drop columns or handle destructive changes; verify the actual table structure directly rather than assuming the entity class is authoritative